Assignment 1: Forward and Backward Propagation in a Simple Neural Network

Course: Deep Learning for Perception
Total Marks: 10

Teacher: Jawwad Ahmed Shamsi, Spring 2025, FAST-NUCES, Karachi

Objective:

The objective of this assignment is to implement forward and backward propagation for a simple neural network with one hidden layer using Python. This will help students understand the core mathematical operations behind neural networks and how gradients are computed for weight updates.

Task Description:

You are required to implement a basic neural network with the following architecture:

- Input layer: 2 neurons
- Hidden layer: 3 neurons (with sigmoid activation)
- Output layer: 1 neuron (with sigmoid activation)

Steps to Follow:

1. Forward Propagation:

- Implement the forward pass using the following equations:

$Z_1 = W_1 X + b_1$

A1 = Sigmoid (Z_1)

$Z_2 = W_2 A_1 + b_2$

A2= Sigmoid (Z_2)

2. Compute Loss:

- Use Binary Cross-Entropy (BCE) loss for classification:

$$Loss = -\frac{1}{m} \sum_{i=1}^{m} \left[Y \log(A_2) + (1 - Y) \log(1 - A_2) \right]$$

3. Backward Propagation:

- Compute gradients for weight updates using the chain rule:
- **Derivative of BCE Loss:**
  - $\frac{\partial Loss}{\partial A_2} = -\frac{Y}{A_2} + \frac{1-Y}{1-A_2}$
- Since $A_2 = \sigma(Z_2)$ and $\sigma'(Z_2) = A_2(1 - A_2)$, the simplified derivative is:
  - $dZ_2 = A_2 - Y$
- **Gradient calculations:**
  - $dW_2 = \frac{1}{m} dZ_2 A_1^T, db_2 = \frac{1}{m} \sum dZ_2$
  - $dA_1 = W_2^T dZ_2$
  - $dZ_1 = dA_1 * Sigmoid'(Z_1)$
  - $dW_1 = \frac{1}{m} dZ_1 X^T, db_1 = \frac{1}{m} \sum dZ_1$

4. Update Weights:

Use gradient descent with a learning rate =0.5:

Use gradient descent with a learning rate α:

- $W_1 = W_1 - \alpha dW_1$
- $b_1 = b_1 - \alpha db_1$
- $W_2 = W_2 - \alpha dW_2$
- $b_2 = b_2 - \alpha db_2$

Implementation Guidelines:

1. Initialize the weights and biases randomly.
2. Use NumPy for matrix operations.
3. Implement functions for forward propagation, loss computation, backward propagation, and parameter updates.
4. Run the model for multiple iterations to observe loss reduction.
5. Test the model using sample input data.
6. Document your code with comments explaining each step.